

DATA STRUCTURES AND ALGORITHMS

23CSE203

LAB MANUAL

SUBMITTED BY	
NAME	Kantamaneni Sree Harshith
ROLL NO	AV.SC.U4CSE25204
YEAR/SEM/SECTION	2ND YEAR/SEM-3/CSE-C
CREDITS	05

SUBMITTED TO	Dr Gayathri Parasa
DESIGNATION	Asst. Professor (Sr. Gr)
DEPARTMENT	SCHOOL OF COMPUTING

INDEX

S.NO	TITLE	PAGE NO
1.	**WEEK 1:** **1(A).** A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1, followed by "Launch!". **1(B).** Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n(power) where: P= principal growth factor, n= number of years. **1(C).** A HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. **1(D).** In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number. **1(E).** Write a recursive python program to print the first n terms of the Fibonacci series.	5-10
2.	**WEEK 2:** **2(A).** Write a python program to implement linear search. **2(B).** Write a python program to implement binary search with and without sorted array as an input.	11-14
3.	**WEEK-3:** **3(A).** Write a python program to implement bubble sort. **3(B).** Write a python program to implement insertion sort. **3(C).** Write a python program to implement selection sort.	15-19
4.	**WEEK-4:** **4(A).** Write a python program to implement quick sort. **4(B).** Write a python program to implement merge sort.	20-23
5.	**WEEK-5:** **5(A).** Write a python program to implement a Singly Linked List with the below operations. 1) Create Linked List 2) Insert Beginning 3) Insert End 4) Insert at Index	24-30

	5) Delete by Value 6) Delete First Node 7) Delete Last Node 8) Count number of nodes 9) Sort the data of SLL 10) Search data in SLL 11) Display or Traverse 12) Exit	
6.	**WEEK-6:** Write a python program to implement a Double Linked List with the below operations. 1) Create Linked List 2) Insert Beginning 3) Insert End 4) Insert at Index 5) Delete by Value 6) Delete First Node 7) Delete Last Node 8) Count number of nodes 9) Display or Traverse 10) Exit	31-39
7.	**WEEK-7:** **7(A).** Write a python program to implement stack operations such as push, pop, peek, display using an array. **7(B).** Write a python program to implement stack operations such as push, pop, peek, display using a linked list.	40-45
8.	**WEEK-8:** **8(A).** Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures. **8(B).** Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.	46-51
9.	**WEEK-9:** Write a python program to implement a Binary Search Tree (BST) and perform the following operations. 1. Insertion 2. Deletion	52-59

	3. Tree Traversals – in-order, pre-order and post-order 4. Count Leaf Nodes 5. Count Internal Nodes 6. Height of the Tree	
10.	**WEEK-10:** Write a menu driven python program to implement an AVL Tree and perform the following operations. 1. Insertion 2. Deletion 3. Tree Traversals 4. Count Leaf Nodes 5. Count Internal Nodes 6. Height of the Tree	60-66
11.	**WEEK-11:** Write a menu-driven Python program to implement a Binary Heap (Max Heap and Min Heap) and perform the following operations: 1. Insertion of an element into the Heap. 2. Deletion of the root element from the Heap. 3. Display the array representation of the Heap. 4. Exit the program.	67-69
12.	**WEEK-12:** Write a program to implement the following three collision resolution techniques in a Hash Table: 1. Separate Chaining 2. Linear Probing 3. Quadratic Probing	70-77

WEEK 1:

1A:

Code:

```
def countdown(n):
    if n == 0:
        print("Launch!")
    else:
        print(n)
        countdown(n - 1)


n = int(input("Enter n: "))
countdown(n)
```

Output:

```
    Python 3.14.0 (v3.14.0:ebf955df7a8, Oct  7 2025, 08:20:14) [Clang 16.0.0 (clang-1600.0.26.6)
    ] on darwin
    Enter "help" below or click "Help" above for more information.
>>>
    ======== RESTART: /Users/k.sreeharshithchowdary/Desktop/ DSA Prog/1_a.py =======
    Enter n: 5
    5
    4
    3
    2
    1
    Launch!
>>>
```

Ln: 12 Col: 0

1B:

Code:

```
def power(p, n):
    if n == 0:
        return 1
    else:
        return p * power(p, n - 1)

p = int(input("Enter principal growth factor: "))
n = int(input("Enter number of years: "))

print("Result:", power(p, n))
```

Output:

```
    Python 3.14.0 (v3.14.0:ebf955df7a8, Oct  7 2025, 08:20:14) [Clang 16.0.0 (clang-1600.0.26.6)
    ] on darwin
    Enter "help" below or click "Help" above for more information.
>>>
    ======== RESTART: /Users/k.sreeharshithchowdary/Desktop/ DSA Prog/1_b.py =======
    Enter principal growth factor: 100
    Enter number of years: 5
    Result: 10000000000
>>>
```

Ln: 8 Col: 0

1C:

Code:

```
def search(arr, key, i):
    if i == len(arr):
        return -1
    if arr[i] == key:
        return i
    return search(arr, key, i + 1)

arr = [101, 102, 103, 104, 105]
key = int(input("Enter employee ID: "))

result = search(arr, key, 0)

if result == -1:
    print("Employee ID not found")
else:
print("Employee ID found at position:", result + 1)
```

Output:



1D:

Code:

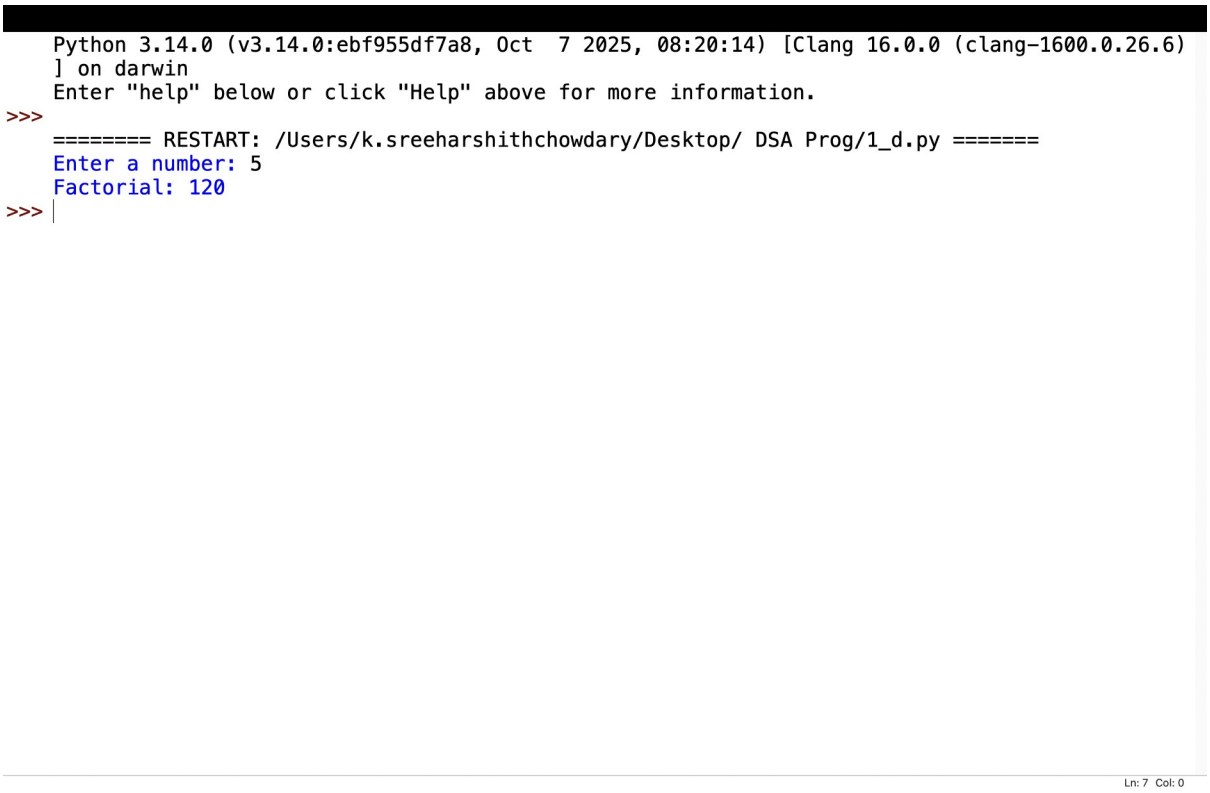
def factorial(n):

```
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)

n = int(input("Enter a number: "))

print("Factorial:", factorial(n))
```

Output:



1E:

Code:

```
def fibonacci(n):
    if n <= 1:
        return n
    else:
        return fibonacci(n-1) + fibonacci(n-2)
n = int(input("Enter number of terms: "))
print("Fibonacci series:")
for i in range(n):
    print(fibonacci(i), end=" ")
```

Output:

```
    Python 3.14.0 (v3.14.0:ebf955df7a8, Oct  7 2025, 08:20:14) [Clang 16.0.0 (clang-1600.0.26.6)
    ] on darwin
    Enter "help" below or click "Help" above for more information.
>>>
    ======== RESTART: /Users/k.sreeharshithchowdary/Desktop/ DSA Prog/1_e.py =======
    Enter number of terms: 5
    Fibonacci series:
    0 1 1 2 3
>>>
```

Ln: 8 Col: 0

WEEK 2:

2A:

Code:

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

# Taking input
n = int(input("Enter the number of elements: "))

arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

key = int(input("Enter the element to search: "))

result = linear_search(arr, key)

if result != -1:
    print("Element found at index", result)
else:
    print("Element not found")
```

Output:

```
    Python 3.14.0 (v3.14.0:ebf955df7a8, Oct  7 2025, 08:20:14) [Clang 16.0.0 (clang-1600.0.26.6)
    ] on darwin
    Enter "help" below or click "Help" above for more information.
>>>
    ======== RESTART: /Users/k.sreeharshithchowdary/Desktop/ DSA Prog/2_a.py =======
    Enter the number of elements: 4
    Enter the elements:
    12
    13
    1
    10
    Enter the element to search: 9
    Element not found
>>>
```

Ln: 13 Col: 0

2B:

Code:

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid

        elif arr[mid] < key:
            low = mid + 1
```

```
        else:
            high = mid - 1

    return -1
# Taking input
n = int(input("Enter number of elements: "))
arr = []
print("Enter elements in sorted order:")
arr = list(map(int, input().split(" ")))
key = int(input("Enter the element to search: "))
result = binary_search(arr, key)
if result != -1:
    print(f"Requested {key} found at index", result)
else:
    print(f"Requested {key} not found")
```

Output:



WEEK 3:

3A:

Code:

```
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

    return arr
# Taking input
n = int(input("Enter number of elements: "))

arr = []
print("Enter the elements:")
arr = list(map(int, input().split()))

result = bubble_sort(arr)

print("Sorted array:", result)
```

Output:



3B:

Code:

```
def insertion_sort(arr):
    n = len(arr)
    for i in range(1,n):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j+1] = arr[j]
            j = j - 1
        arr[j+1] = key
    return arr
```

```
n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    element = int(input())
    arr.append(element)

result = insertion_sort(arr)
print("Elements after Insertion Sort:")
print(result)
```

Output:

```
Python 3.14.0 (v3.14.0:ebf955df7a8, Oct  7 2025, 08:20:14) [Clang 16.0.0 (clang-1600.0.26.6)
] on darwin
Enter "help" below or click "Help" above for more information.
>>>
======== RESTART: /Users/k.sreeharshithchowdary/Desktop/ DSA Prog/3_b.py =======
Enter the number of elements: 4
Enter the elements:
12
34
2
11
Elements after Insertion Sort:
[2, 11, 12, 34]
>>>
```

Ln: 13 Col: 0

3C:

Code:

```
def selection_sort(arr):
    n=len(arr)
    for i in range(n-1):
        min_index=i
        for j in range (i+1,n):
            if arr[j]<arr[min_index]:
                min_index=j
        arr[i],arr[min_index]=arr[min_index],arr[i]
    return arr

n=int(input("Enter the number of elements: "))
arr=[]
print("Enter the elements:")
for i in range(n):
    element=int(input())
    arr.append(element)

result=selection_sort(arr)
print("Elements after Selection Sort:")
print(result)
```

Output:

```
    Python 3.14.0 (v3.14.0:ebf955df7a8, Oct  7 2025, 08:20:14) [Clang 16.0.0 (clang-1600.0.26.6)
    ] on darwin
    Enter "help" below or click "Help" above for more information.
>>>
    ======== RESTART: /Users/k.sreeharshithchowdary/Desktop/ DSA Prog/3_c.py =======
    Enter the number of elements: 4
    Enter the elements:
    12
    23
    3
    4
    Elements after Selection Sort:
    [3, 4, 12, 23]
>>>
```

Ln: 13 Col: 0

WEEK 4:

4A:

Code:

```
def quicksort(a, low, high):
    if low < high:
        i = low
        j = high
        pivot = low

        while i < j:
            while i < len(a) and a[i] <= a[pivot]:
                i += 1
            while a[j] > a[pivot]:
                j -= 1
            if i < j:
                a[i], a[j] = a[j], a[i]
        a[j], a[pivot] = a[pivot], a[j]

        quicksort(a, low, j - 1)
        quicksort(a, j + 1, high)

a = list(map(int, input("Enter numbers to sort:").split(" ")))
n = len(a)
quicksort(a, 0, n - 1)
print("Sorted array:")
print(a)
```

Output:



4B:

Code:

```
def merging(arr, low, mid, high):
    result = []
    i = low
    j = mid + 1
    while i <= mid and j <= high:
        if arr[i] < arr[j]:
            result.append(arr[i])
            i += 1
        else:
            result.append(arr[j])
            j += 1
```

```
    while i <= mid:
        result.append(arr[i])
        i += 1
    while j <= high:
        result.append(arr[j])
        j += 1

    i = low
    j = 0
    while i <= high:
        arr[i] = result[j]
        i += 1
        j += 1


def merge_sort(arr, low, high):
    if low < high:
        mid = (low + high) // 2
        merge_sort(arr, low, mid)
        merge_sort(arr, mid + 1, high)
        merging(arr, low, mid, high)


arr = list(map(int, input("Enter numbers to sort: ").split()))
n = len(arr)
merge_sort(arr, 0, n - 1)
```

```
print("Sorted array:", arr)
```

Output:



WEEK 5:

5A:

Code:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None



class LinkedList:
    def __init__(self):
        self.head = None

    def create(self, n):
        self.head = None

        for i in range(n):
            data = int(input(f"Enter data for node {i + 1}: "))
            self.insert_end(data)

        print("Linked List created successfully")

    def insert_begin(self, data):
        new = Node(data)
        new.next = self.head
        self.head = new
```

```
def insert_end(self, data):
    new = Node(data)

    if self.head is None:
        self.head = new
    else:
        temp = self.head

        while temp.next:
            temp = temp.next

        temp.next = new

def insert_position(self, data, position):
    new = Node(data)

    if position < 1:
        print("Invalid position")
        return

    if position == 1:
        new.next = self.head
        self.head = new
        print("Node inserted successfully")
        return
```

```
        temp = self.head

        for i in range(position - 2):
            if temp is None:
                print("Invalid position")
                return
            temp = temp.next

        if temp is None:
            print("Invalid position")
        else:
            new.next = temp.next
            temp.next = new
            print("Node inserted successfully")

    def delete_value(self, value):
        if self.head is None:
            print("No Data to delete")
            return

        if self.head.data == value:
            self.head = self.head.next
            print("Value deleted")
            return
```

```
        temp = self.head

        while temp.next and temp.next.data != value:
            temp = temp.next

        if temp.next is None:
            print("Value not present")
        else:
            temp.next = temp.next.next
            print("Value deleted")

    def delete_first(self):
        if self.head is None:
            print("No Data to delete")
        else:
            print("Deleted Value =", self.head.data)
            self.head = self.head.next

    def delete_last(self):
        if self.head is None:
            print("No Data to delete")

        elif self.head.next is None:
            print("Deleted Value =", self.head.data)
            self.head = None
```

```
    else:
        temp = self.head

        while temp.next.next:
            temp = temp.next

        print("Deleted Value =", temp.next.data)
        temp.next = None

def count(self):
    if self.head is None:
        print("No Linked List")
    else:
        c = 0
        temp = self.head

        while temp:
            c += 1
            temp = temp.next

        print("Number of nodes =", c)

def display(self):
    if self.head is None:
        print("No Linked List")
    else:
```

```
        temp = self.head

        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next

        print("None")


ll = LinkedList()

while True:
    print("\n--- Singly Linked List Operations ---")
    print("1. Create a Linked List")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Specific Position")
    print("5. Delete a Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Number of Nodes")
    print("9. Display Nodes")
    print("10. Exit")

    osition = int(input("Enter position: "))
        ll.insert_position(data, position)
```

```
elif choice == 5:
    value = int(input("Enter value to delete: "))
    ll.delete_value(value)

elif choice == 6:
    ll.delete_first()

elif choice == 7:
    ll.delete_last()choice = int(input("Enter your choice: ")

elif choice == 8:
    ll.count()

elif choice == 9:
    ll.display()

elif choice == 10:
    print("Program ended")
    break

else:
    print("Invalid choice")
```

Output:

```
9. Display Nodes
10. Exit
Enter your choice: 2
Enter data: 22

--- Singly Linked List Operations ---
1. Create a Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Position
5. Delete a Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display Nodes
10. Exit
Enter your choice: 9
22 -> 10 -> 20 -> 4 -> None

--- Singly Linked List Operations ---
1. Create a Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Specific Position
5. Delete a Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display Nodes
10. Exit
Enter your choice:
```

Ln: 50 Col: 8

WEEK 6:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None


class DoublyLinkedList:
    def __init__(self):
        self.head = None

    def create(self):
        n = int(input("Enter number of nodes: "))

        for i in range(n):
            data = int(input("Enter data: "))
            new_node = Node(data)

            if self.head is None:
                self.head = new_node
            else:
                temp = self.head

                while temp.next is not None:
                    temp = temp.next

                temp.next = new_node
                new_node.prev = temp
```

```
    print("Linked List created successfully.")

def insert_beginning(self):
    data = int(input("Enter data: "))
    new_node = Node(data)

    if self.head is None:
        self.head = new_node
    else:
        new_node.next = self.head
        self.head.prev = new_node
        self.head = new_node

    print("Node inserted at beginning.")

def insert_end(self):
    data = int(input("Enter data: "))
    new_node = Node(data)

    if self.head is None:
        self.head = new_node
    else:
        temp = self.head

        while temp.next is not None:
            temp = temp.next

        temp.next = new_node
        new_node.prev = temp
```

```
        print("Node inserted at end.")

    def insert_position(self):
        position = int(input("Enter position: "))
        data = int(input("Enter data: "))

        if position <= 0:
            print("Invalid position.")
            return

        if position == 1:
            new_node = Node(data)

            if self.head is not None:
                new_node.next = self.head
                self.head.prev = new_node

            self.head = new_node
            print("Node inserted.")
            return

        temp = self.head

        for i in range(1, position - 1):
            if temp is None:
                print("Invalid position.")
                return
            temp = temp.next

        if temp is None:
```

```
            print("Invalid position.")
            return

        new_node = Node(data)

        new_node.next = temp.next
        new_node.prev = temp

        if temp.next is not None:
            temp.next.prev = new_node

        temp.next = new_node

        print("Node inserted.")

    def delete_beginning(self):
        if self.head is None:
            print("Linked List is empty.")
            return

        self.head = self.head.next

        if self.head is not None:
            self.head.prev = None

        print("Node deleted from beginning.")

    def delete_end(self):
        if self.head is None:
            print("Linked List is empty.")
```

```
            return

        temp = self.head

        if temp.next is None:
            self.head = None
            print("Node deleted from end.")
            return

        while temp.next is not None:
            temp = temp.next

        temp.prev.next = None

        print("Node deleted from end.")

    def delete_position(self):
        position = int(input("Enter position: "))

        if self.head is None:
            print("Linked List is empty.")
            return

        if position <= 0:
            print("Invalid position.")
            return

        if position == 1:
            self.head = self.head.next
```

```
            if self.head is not None:
                self.head.prev = None

            print("Node deleted.")
            return

        temp = self.head

        for i in range(1, position):
            if temp is None:
                print("Invalid position.")
                return
            temp = temp.next

        if temp is None:
            print("Invalid position.")
            return

        if temp.next is not None:
            temp.next.prev = temp.prev

        if temp.prev is not None:
            temp.prev.next = temp.next

        print("Node deleted.")

    def count(self):
        count = 0
        temp = self.head
```

```
        while temp is not None:
            count += 1
            temp = temp.next

        print("Number of nodes:", count)

    def display(self):
        if self.head is None:
            print("Linked List is empty.")
            return

        temp = self.head
        print("Doubly Linked List:")

        while temp is not None:
            print(temp.data, end=" <-> ")
            temp = temp.next

        print("None")


dll = DoublyLinkedList()

while True:
    print("1. Create a LL")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific position")
    print("5. Deletion at beginning")
    print("6. Deletion at end")
```

```
print("7. Deletion at specific position")
print("8. Count no. of nodes")
print("9. Display / Traverse")
print("10. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
    dll.create()

elif choice == 2:
    dll.insert_beginning()

elif choice == 3:
    dll.insert_end()

elif choice == 4:
    dll.insert_position()

elif choice == 5:
    dll.delete_beginning()

elif choice == 6:
    dll.delete_end()

elif choice == 7:
    dll.delete_position()

elif choice == 8:
    dll.count()
```

```
elif choice == 9:
    dll.display()

elif choice == 10:
    print("Program exited.")
    break

else:
    print("Invalid choice. Please try again.")
```

WEEK 7:

7A:

Code:

```
class StackArray:
    def __init__(self, size):
        self.size = size
        self.stack = [None] * size
        self.top = -1

    def push(self, item):
        if self.top == self.size - 1:
            print("Stack Overflow")
        else:
            self.top += 1
            self.stack[self.top] = item
            print(item, "pushed into the stack")

    def pop(self):
        if self.top == -1:
            print("Stack Underflow")
        else:
            item = self.stack[self.top]
            self.stack[self.top] = None
            self.top -= 1
            print(item, "popped from the stack")

    def peek(self):
        if self.top == -1:
            print("Stack is empty")
        else:
```

```
            print("Top element:", self.stack[self.top])

    def display(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("Stack elements (top to bottom):")
            for i in range(self.top, -1, -1):
                print(self.stack[i])



size = int(input("Enter size of stack: "))
s = StackArray(size)

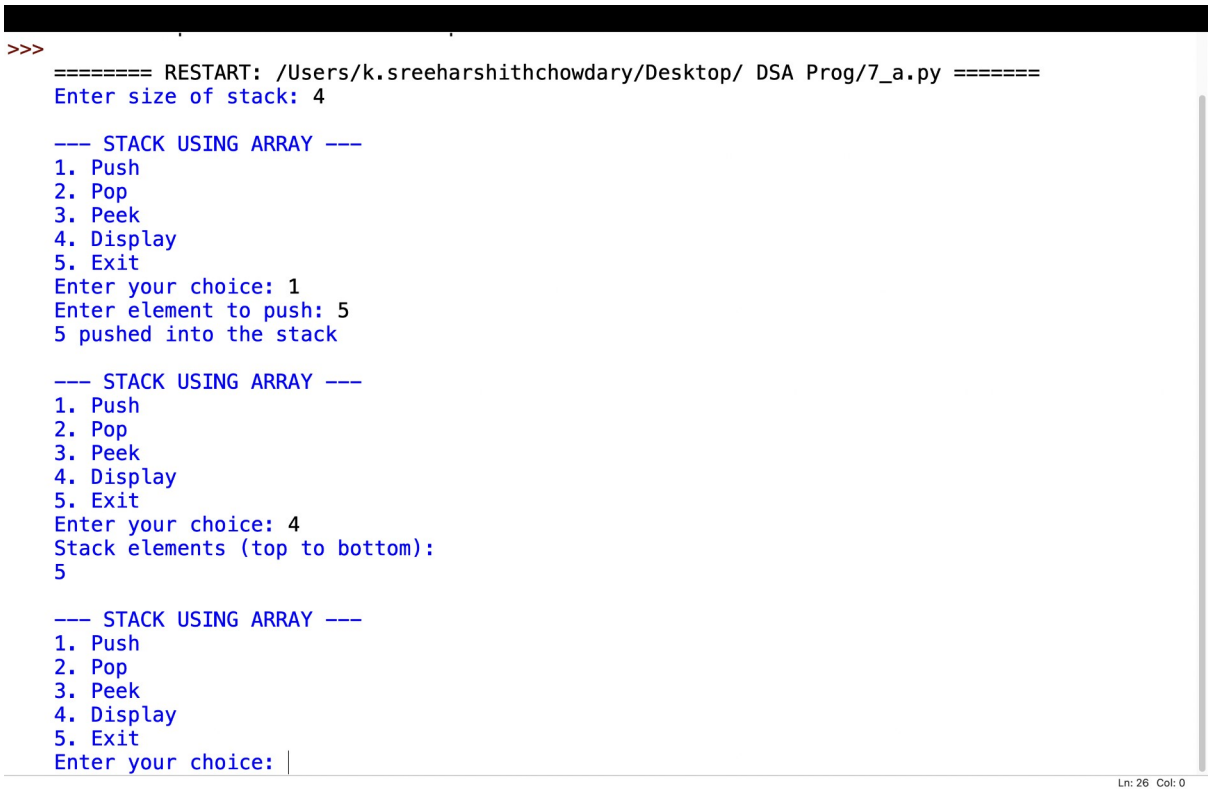
while True:
    print("\n--- STACK USING ARRAY ---")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter element to push: "))
        s.push(item)
    elif choice == 2:
        s.pop()
    elif choice == 3:
```

```
        s.peek()
    elif choice == 4:
        s.display()
    elif choice == 5:
        print("Exiting program...")
        break
    else:
        print("Invalid choice!. Try again.")
```

Output:



7B:

Code:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class StackLinkedList:
    def __init__(self):
        self.top = None

    def push(self, data):
        new_node = Node(data)
        new_node.next = self.top
        self.top = new_node
        print(data, "pushed into the stack")

    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            popped = self.top.data
            self.top = self.top.next
            print(popped, "popped from the stack")

    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
```

```
            print("Top element:", self.top.data)

    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Stack elements (top to bottom):")
            temp = self.top
            while temp:
                print(temp.data)
                temp = temp.next


sll = StackLinkedList()

while True:
    print("\n--- STACK USING LINKED LIST ---")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter element to push: "))
        sll.push(item)
    elif choice == 2:
        sll.pop()
```

```
elif choice == 3:
    sll.peek()
elif choice == 4:
    sll.display()
elif choice == 5:
    print("Exiting program...")
    break
else:
    print("Invalid choice!. Try again.")
```

Output:



WEEK 8:

8A:

Code:

class QueueEx:

    def **__init__**(self, size=5):

        self.size = size

        self.queue = [None] * size

        self.front = -1

        self.rear = -1

```
def enqueue(self, x):
    if self.rear == self.size - 1:
        print("Queue Overflow")
    else:
        if self.front == -1:
            self.front = 0

        self.rear += 1
        self.queue[self.rear] = x
        print(f"{x} inserted into the queue")

def dequeue(self):
    if self.front == -1 or self.front > self.rear:
        print("Queue Underflow")
    else:
        x = self.queue[self.front]
        self.queue[self.front] = None
        self.front += 1
        print(f"{x} deleted from the queue")

        if self.front > self.rear:
            self.front = -1
            self.rear = -1

def peek(self):
    if self.front == -1:
        print("Queue is empty")
    else:
        print("Front element:", self.queue[self.front])

def display(self):
    if self.front == -1:
        print("Queue is empty")
    else:
        print("The elements of the queue are:")
        for i in range(self.front, self.rear + 1):
            print(self.queue[i])
```

```
size = int(input("Enter the size of Queue: "))
q = QueueEx(size)
while True:
    print("\n------ Queue Menu ------")
    print("1. Enqueue")
     print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()

    elif choice == 4:
        q.display()

    elif choice == 5:
        print("Exiting Program")
        break

    else:
        print("Invalid choice")
```

Output:

```
Enter the element to enqueue: 23
23 inserted into the queue

------ Queue Menu ------
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 234
234 inserted into the queue

------ Queue Menu ------
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
The elements of the queue are:
23
234

------ Queue Menu ------
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice:
```

Ln: 35 Col: 2

8B:

class Node:

    def **init**(self, data):

        self.data = data

        self.next = None

class CircularQueue:

    def **init**(self, size=5):

        self.size = size

        self.front = None

        self.rear = None

        self.count = 0

```
def enqueue(self, x):
    if self.count == self.size:
        print("Queue Overflow")
    else:
        new_node = Node(x)

        if self.front is None:
```

```
            self.front = new_node
            self.rear = new_node
            new_node.next = new_node
        else:
            new_node.next = self.front
            self.rear.next = new_node
            self.rear = new_node

        self.count += 1
        print(f"{x} inserted into the queue")

def dequeue(self):
    if self.front is None:
        print("Queue Underflow")
    else:
        x = self.front.data

        if self.front == self.rear:
            self.front = None
            self.rear = None
        else:
            self.front = self.front.next
            self.rear.next = self.front

        self.count -= 1
        print(f"{x} deleted from the queue")

def peek(self):
    if self.front is None:
        print("Queue is empty")
    else:
        print("Front element:", self.front.data)

def display(self):
    if self.front is None:
        print("Queue is empty")
    else:
        print("The elements of the queue are:")

        temp = self.front

        while True:
            print(temp.data)
            temp = temp.next

            if temp == self.front:
                break
```

size = int(input("Enter the size of Queue: "))

q = CircularQueue(size)

while True:

print("\n----- Queue Menu -----")

print("1. Enqueue")

print("2. Dequeue")

print("3. Peek")

print("4. Display")

print("5. Exit")

```
choice = int(input("Enter your choice: "))

if choice == 1:
    item = int(input("Enter the element to enqueue: "))
    q.enqueue(item)

elif choice == 2:
    q.dequeue()

elif choice == 3:
    q.peek()

elif choice == 4:
    q.display()

elif choice == 5:
    print("Exiting Program")
    break

else:
    print("Invalid Choice!")
```

Output:

```
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 23
23 inserted into the queue

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
23 deleted from the queue

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue is empty

----- Queue Menu -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice:
```

Ln: 41 Col: 19

WEEK 9:

9A:

Code:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


def insert(root, x):
    if root is None:
        return Node(x)
    if x < root.data:
        root.left = insert(root.left, x)
    elif x > root.data:
        root.right = insert(root.right, x)
    else:
        print("Duplicate value is not allowed.")
    return root



def inorder(root):
    if root is None:
        return
    inorder(root.left)
    print(root.data, end=" ")
    inorder(root.right)

def preorder(root):
    if root is None:
```

```
        return
    print(root.data, end=" ")
    preorder(root.left)
    preorder(root.right)

def postorder(root):
    if root is None:
        return
    postorder(root.left)
    postorder(root.right)
    print(root.data, end=" ")

def find_min(root):
    if root is None:
        return None
    while root.left is not None:
        root = root.left
    return root

def find_max(root):
    if root is None:
        return None
    while root.right is not None:
        root = root.right
    return root

def search(root, data):
    if root is None:
        return None
    if data < root.data:
```

```
        return search(root.left, data)
    elif data > root.data:
        return search(root.right, data)
    else:
        return root

def count_nodes(root):
    if root is None:
        return 0
    return count_nodes(root.left) + count_nodes(root.right) + 1

def count_leaves(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 1
    return count_leaves(root.left) + count_leaves(root.right)

def count_internal(root):
    if root is None:
        return 0
    if root.left is None and root.right is None:
        return 0
    return count_internal(root.left) + count_internal(root.right) + 1

def height(root):
    if root is None:
        return -1
    left_height = height(root.left)
    right_height = height(root.right)
```

```
    return max(left_height, right_height) + 1


def delete(root, x):
    if root is None:
        print("Element not found.")
        return None
    if x < root.data:
        root.left = delete(root.left, x)
    elif x > root.data:
        root.right = delete(root.right, x)
    else:
        # Case 1: Node has no children
        if root.left is None and root.right is None:
            root = None
        # Case 2: Node has only right child
        elif root.left is None:
            root = root.right
        # Case 3: Node has only left child
        elif root.right is None:
            root = root.left
        # Case 4: Node has two children
        else:
            successor = find_min(root.right)
            root.data = successor.data
            root.right = delete(root.right, successor.data)
    return root

def print_level(root, level):
    if root is None:
```

```
        return
    if level == 0:
        print(root.data, end=" ")
    else:
        print_level(root.left, level - 1)
        print_level(root.right, level - 1)


def level_order(root):
    h = height(root)
    for level in range(h):
        print_level(root, level)


# ======================================================
# MAIN PROGRAM
# ======================================================

root = None
while True:
    print("\n\n******** BINARY SEARCH TREE ********")
    print("1. INSERT")
    print("2. PREORDER TRAVERSAL")
    print("3. INORDER TRAVERSAL")
    print("4. POSTORDER TRAVERSAL")
    print("5. FIND MINIMUM")
    print("6. FIND MAXIMUM")
    print("7. SEARCH")
    print("8. DELETE")
    print("9. COUNT NODES")
```

```
print("10. COUNT LEAVES")
print("11. COUNT INTERNAL NODES")
print("12. HEIGHT")
print("13. LEVEL ORDER TRAVERSAL")
print("14. EXIT")

choice = int(input("Enter your choice: "))
if choice == 1:
    data = int(input("Enter the data to be inserted: "))
    root = insert(root, data)
    print("Element inserted successfully.")
elif choice == 2:
    print("Preorder Traversal:", end=" ")
    if root is None:
        print("Tree is empty.")
    else:
        preorder(root)
        print()
elif choice == 3:
    print("Inorder Traversal:", end=" ")
    if root is None:
        print("Tree is empty.")
    else:
        inorder(root)
        print()
elif choice == 4:
    print("Postorder Traversal:", end=" ")
    if root is None:
        print("Tree is empty.")
    else:
```

```
            postorder(root)
            print()
        elif choice == 5:
            node = find_min(root)
            if node is None:
                print("Tree is empty.")
            else:
                print("Minimum element:", node.data)
        elif choice == 6:
            node = find_max(root)
            if node is None:
                print("Tree is empty.")
            else:
                print("Maximum element:", node.data)
        elif choice == 7:
            data = int(input("Enter the data to be searched: "))
            node = search(root, data)
            if node is None:
                print("Element", data, "is not found.")
            else:
                print("Element", data, "is found.")
        elif choice == 8:
            data = int(input("Enter the data to be deleted: "))
            root = delete(root, data)
        elif choice == 9:
            print("Number of nodes:", count_nodes(root))
        elif choice == 10:
            print("Number of leaf nodes:", count_leaves(root))
        elif choice == 11:
            print("Number of internal nodes:", count_internal(root))
```

```
    elif choice == 12:
        print("Height of BST:", height(root))
    elif choice == 13:
        print("Level Order Traversal:", end=" ")
        if root is None:
            print("Tree is empty.")
        else:
            level_order(root)
            print()
    elif choice == 14:
        print("Program terminated.")
        break
    else:
        print("Invalid choice. Please try again.")
```

Output:

```
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice: 2
Preorder Traversal: 23 234


******** BINARY SEARCH TREE ********
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. FIND MINIMUM
6. FIND MAXIMUM
7. SEARCH
8. DELETE
9. COUNT NODES
10. COUNT LEAVES
11. COUNT INTERNAL NODES
12. HEIGHT
13. LEVEL ORDER TRAVERSAL
14. EXIT
Enter your choice:
```

Ln: 63 Col: 22

WEEK 10:

10A:

Code:

```
class Node:
    def __init__(self, x):
        self.left = None
        self.data = x
        self.right = None
        self.h = 0

#AVL TREE Program
def height(root):
    if root is None:
        return -1
    else:
        return root.h

# LL Rotation
def LL(x):
    y = x.left
    x.left = y.right
    y.right = x
    x.h = max(height(x.left), height(x.right)) + 1
    y.h = max(height(y.left), height(y.right)) + 1
    return y

# RR Rotation
def RR(x):
    y = x.right
    x.right = y.left
```

```
    y.left = x
    x.h = max(height(x.left), height(x.right)) + 1
    y.h = max(height(y.left), height(y.right)) + 1
    return y

# LR Rotation
def LR(k3):
    k3.left = RR(k3.left)
    return LL(k3)

# RL Rotation
def RL(k3):
    k3.right = LL(k3.right)
    return RR(k3)

# Inorder Traversal
def inorder(root):
    if root is not None:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)


# Preorder Traversal
def preorder(root):
    if root is not None:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)
```

```
# Postorder Traversal
def postorder(root):
    if root is not None:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")

# Insert a node into AVL Tree
def insert(root, x):
    if root is None:
        return Node(x)

    if x < root.data:
        root.left = insert(root.left, x)
        if height(root.left) - height(root.right) == 2:
            if x < root.left.data:
                root = LL(root)
            else:
                root = LR(root)

    elif x > root.data:
        root.right = insert(root.right, x)
        if height(root.right) - height(root.left) == 2:
            if x > root.right.data:
                root = RR(root)
            else:
                root = RL(root)
    else:
        print("Duplicates not allowed")
```

```
    root.h = max(height(root.left),height(root.right)) + 1
    return root


# Delete a node from AVL Tree
def delete(root, x):
    # Element not found
    if root is None:
        return None

    # Search in left subtree
    if x < root.data:
        root.left = delete(root.left, x)
        if height(root.left) - height(root.right) == 2:
            if x < root.left.data:
                root = LL(root)
            else:
                root = LR(root)

    # Search in right subtree
    elif x > root.data:
        root.right = delete(root.right, x)
        if height(root.right) - height(root.left) == 2:
            if x > root.right.data:
                root = RR(root)
            else:
                root = RL(root)

    # When target found
    else:
```

```
        if root.left is None and root.right is None:
            root = None
        elif root.right is None:
            root = root.left
        elif root.left is None:
            root = root.right
        else:
            successor = root.right   # Find inorder successor
            while successor.left is not None:
                successor = successor.left
            root.data = successor.data   #Copy successor data
            root.right = delete(root.right, successor.data)     # Delete the inorder successor

        root.h = max(height(root.left), height(root.right)) + 1
    return root

# ------------------------------
# Main Program
# ------------------------------

root = None

while True:

    print("\n***** AVL TREE MENU *****")
    print("1. INSERT")
    print("2. PREORDER TRAVERSAL")
    print("3. INORDER TRAVERSAL")
    print("4. POSTORDER TRAVERSAL")
    print("5. DELETE")
```

```
print("6. EXIT")

choice = int(input("Enter your choice: "))
if choice == 1:
    data = int(input("Enter the data to be inserted: "))
    root = insert(root, data)
elif choice == 2:
    print("Preorder Traversal:")
    preorder(root)
    print()
elif choice == 3:
    print("Inorder Traversal:")
    inorder(root)
    print()
elif choice == 4:
    print("Postorder Traversal:")
    postorder(root)
    print()
elif choice == 5:
    data = int(input("Enter the data to be deleted: "))
    root = delete(root, data)
elif choice == 6:
    print("Program terminated.")
    break
else:
    print("Invalid choice!")
```

Output:

```
Enter the data to be inserted: 12

***** AVL TREE MENU *****
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. DELETE
6. EXIT
Enter your choice: 1
Enter the data to be inserted: 4

***** AVL TREE MENU *****
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. DELETE
6. EXIT
Enter your choice: 2
Preorder Traversal:
12 4

***** AVL TREE MENU *****
1. INSERT
2. PREORDER TRAVERSAL
3. INORDER TRAVERSAL
4. POSTORDER TRAVERSAL
5. DELETE
6. EXIT
Enter your choice:
```

Ln: 44 Col: 19

WEEK 11:

11A:

Code:

```
def insert(heap, value):
    heap.append(value)
    i = len(heap) - 1
    while i > 0:
        parent = (i - 1) // 2
        if heap[i] > heap[parent]:
            heap[i], heap[parent] = heap[parent], heap[i]
            i = parent
        else:
            break

def delete(heap):
    if len(heap) == 0:
        print("Heap is empty")
        return
    print("Deleted element:", heap[0])
    heap[0] = heap[-1]
    heap.pop()

    n = len(heap)
    i = 0
    # Move the root downward
    while True:
        largest = i
        left = 2 * i + 1
        right = 2 * i + 2
```

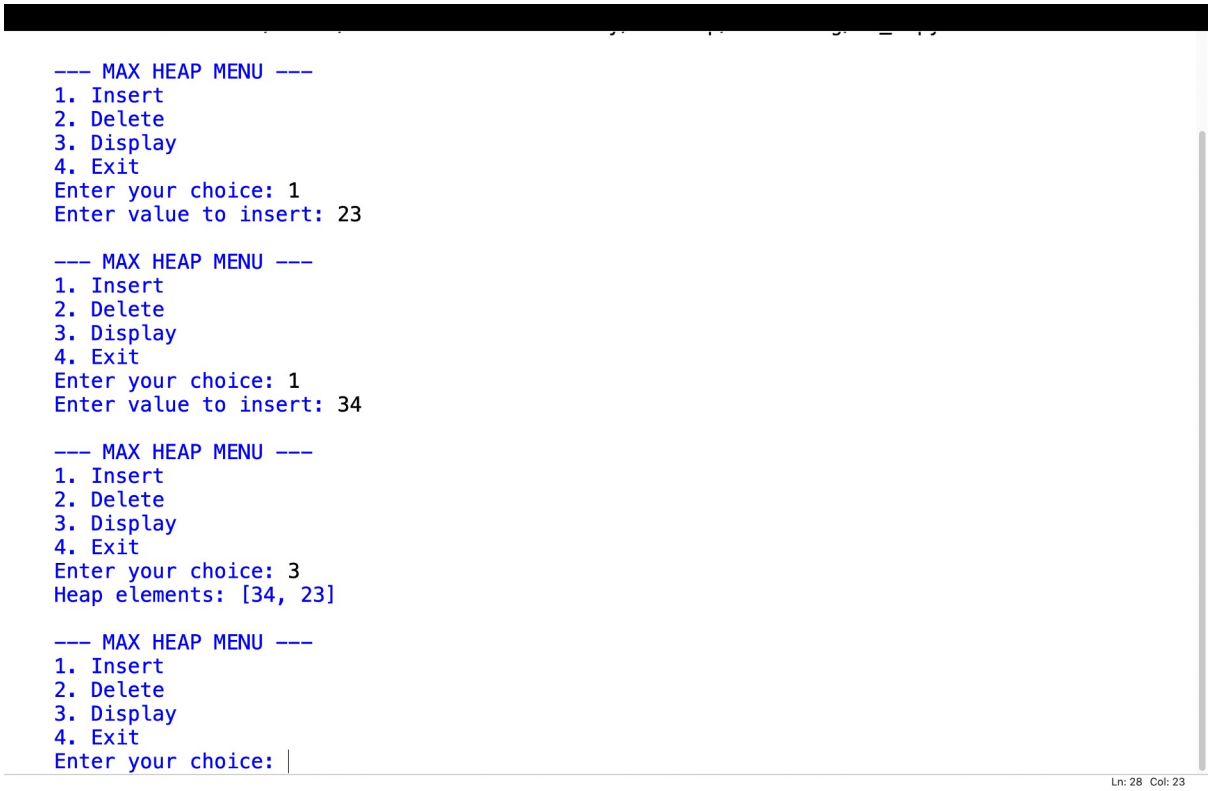
```
        if left < n and heap[left] > heap[largest]:
            largest = left
        if right < n and heap[right] > heap[largest]:
            largest = right
        if largest != i:
            heap[i], heap[largest] = heap[largest], heap[i]
            i = largest
        else:
            break

def display(heap):
    if len(heap) == 0:
        print("Heap is empty")
    else:
        print("Heap elements:", heap)

#Main Program
heap = []
while True:
    print("\n--- MAX HEAP MENU ---")
    print("1. Insert")
    print("2. Delete")
    print("3. Display")
    print("4. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        value = int(input("Enter value to insert: "))
        insert(heap, value)
    elif choice == 2:
        delete(heap)
```

```
    elif choice == 3:
        display(heap)
    elif choice == 4:
        print("Exiting program")
        break
    else:
        print("Invalid choice")
```

Output:



WEEK 12:

12A: Separate Chaining

Code:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

SIZE = 10
ht = [None] * SIZE



def insertValue(value):
    index = value % SIZE
    newnode = Node(value)
    if ht[index] is None:
        ht[index] = newnode
    else:
        temp = ht[index]
        while temp.next is not None:
            temp = temp.next
        temp.next = newnode



def displayValue():
    for i in range(SIZE):
        print(i, end=": ")
        temp = ht[i]
        if temp is None:
            print("None")
```

```
        else:
            while temp is not None:
                print(temp.data, end=" -> ")
                temp = temp.next
            print("None")


repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
    print("3. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
        value = int(input("Enter value: "))
        insertValue(value)
    elif ch == 2:
        print("\nHash table values:")
        displayValue()
    elif ch == 3:
        repeat = False
    else:
        print("Invalid choice")
```

Output:

```
3. Exit
Enter your choice: 1
Enter value: 23

1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter value: 4

1. Insert
2. Display
3. Exit
Enter your choice: 2

Hash table values:
0: None
1: None
2: None
3: 23 -> None
4: 4 -> None
5: None
6: None
7: None
8: None
9: None

1. Insert
2. Display
3. Exit
Enter your choice:
```

Ln: 25 Col: 0

12 B:Linear Probing

Code:

```
SIZE = 10
a = [-1] * SIZE
def insertValue(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == -1:
            a[index] = x
            print("Element inserted")
            return
    print("Hash table is full")



def deleteValue(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == x:
            a[index] = -1
            print("Element deleted")
            return
    print("Element not found")

def search(x):
    for i in range(SIZE):
        index = ((x % SIZE) + i) % SIZE
        if a[index] == x:
            print("Element found at index", index)
            return
        if a[index] == -1:
```

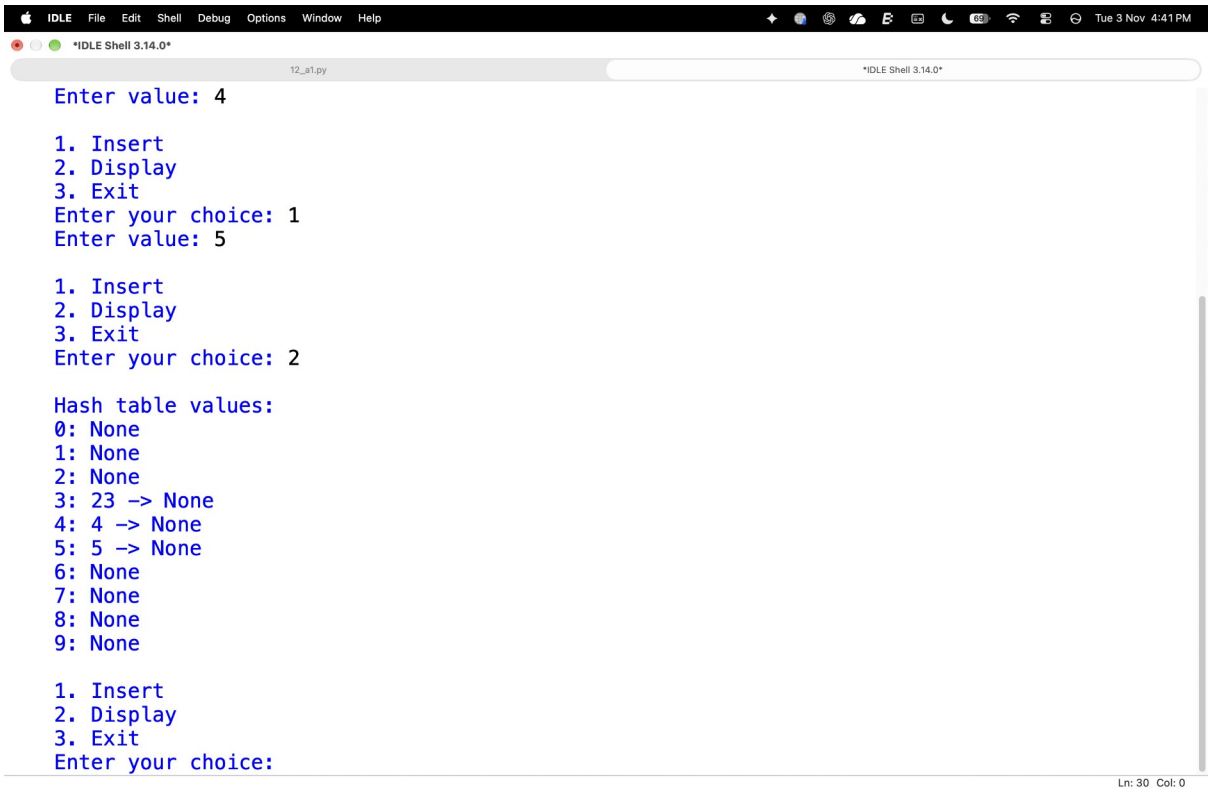
```
            break
    print("Element not found")


def displayValue():
    print("Index\tValue")
    for i in range(SIZE):
        print(i, "\t", a[i])

# Menu-driven program
repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
    print("3. Delete")
    print("4. Search")
    print("5. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
        value = int(input("Enter value: "))
        insertValue(value)
    elif ch == 2:
        print("Values are:")
        displayValue()
    elif ch == 3:
        value = int(input("Enter value to delete: "))
        deleteValue(value)
    elif ch == 4:
        value = int(input("Enter value to search: "))
        search(value)
```

```
elif ch == 5:
    repeat = False
else:
    print("Invalid choice")
```

Output:



12 C:Quadratic Probing

Code:

```
SIZE = 10
a = [-1] * SIZE

def insertValue(value):
    for i in range(SIZE):
        index = ((value % SIZE) + i * i) % SIZE
    if a[index] == -1:
            a[index] = value
            print("Element inserted at index", index)
            return
    print("No empty slot found in the probe sequence")



def displayValue():
    print("Values are:")
    for i in range(SIZE):
        print(a[i])

# Menu-driven program
repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
    print("3. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
        value = int(input("Enter value: "))
        insertValue(value)
```

```
elif ch == 2:
    displayValue()
elif ch == 3:
    repeat = False
else:
    print("Invalid choice")
```

Output:

```
Enter your choice: 1
Enter value: 3
Element inserted at index 4

1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter value: 4
Element inserted at index 5

1. Insert
2. Display
3. Exit
Enter your choice: 2
Values are:
-1
-1
-1
2
3
4
-1
-1
-1
-1

1. Insert
2. Display
3. Exit
Enter your choice:
```

Ln: 33 Col: 0